

一、单项选择题

1.B 2.B 3.C 4.C 5.C 6.A 7.C 8.C 9.D 10.C
11.A 12.C 13.C 14.D 15.B

二、判断题

1.× 2.× 3.× 4.√ 5.× 6.× 7.× 8.× 9.× 10.× 11.√ 12.√ 13.× 14.
× 15.×

三、填空题

1. 数据元素
2. 假溢出
3. 对数据的运算
4. 顺序
5. 3
6. $O(n^2)$
7. 队列
8. $2n-1$
9. $n-i+1$ 、 $n-i$
10. ABCDE
11. 2
12. (30, 20, 22, 16, 18, 19)
13. 出度、入度

四、简答题

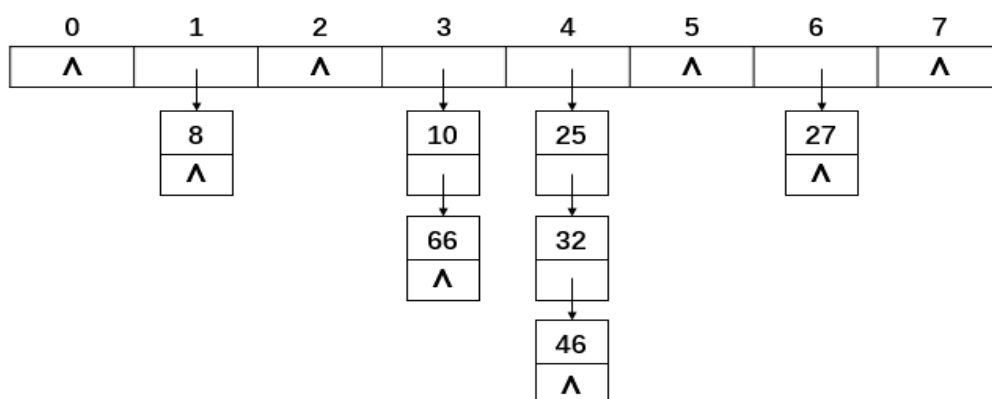
1. (1) 线性探测法

地址	0	1	2	3	4	5	6	7
关键字	46	8		10	25	66	27	32

$$ASL_1 = (1+1+1+3+1+4+5)/7 = 16/7$$

$$ASL_2 = (3+2+1+8+7+6+5)/7 = 32/7$$

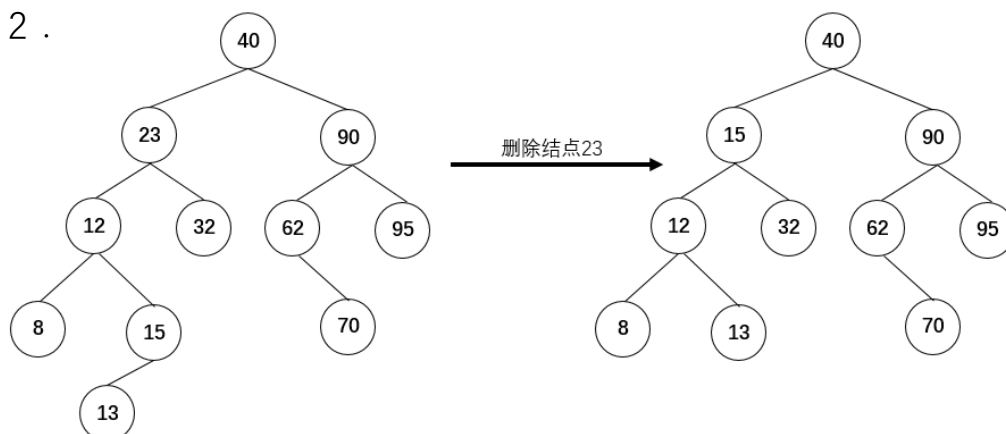
(2) 链地址法



$$ASL_1 = (1+1+2+1+2+3+1)/7 = 11/7$$

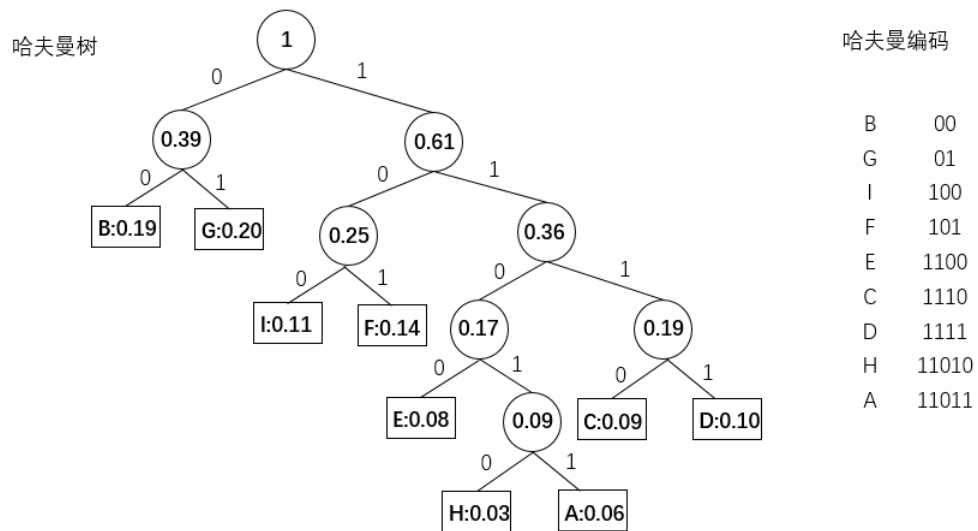
$$ASL_2 = (1+2+1+3+4+1+2)/7 = 14/7$$

2 .



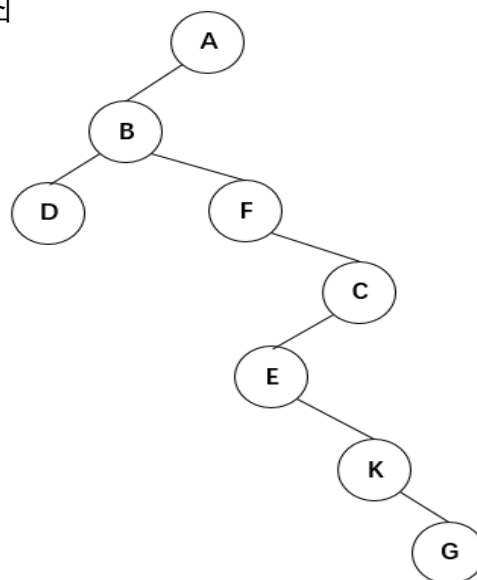
注：删除一个结点后可由中序序列的直接前驱或直接后继代替

3 .

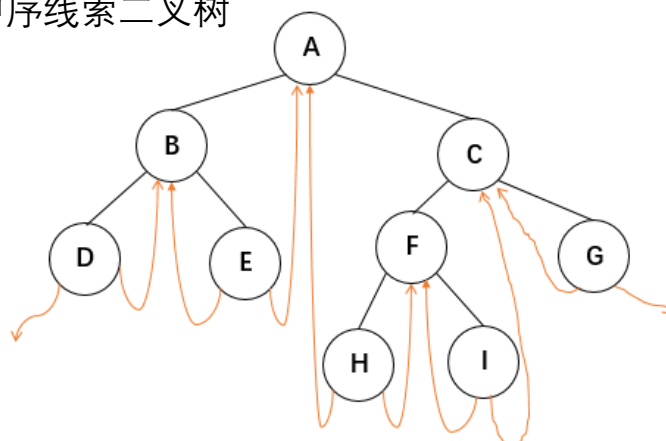


4.后序遍历序列: D B F E K G C A

转换为二叉树如右图



5. 中序线索二叉树

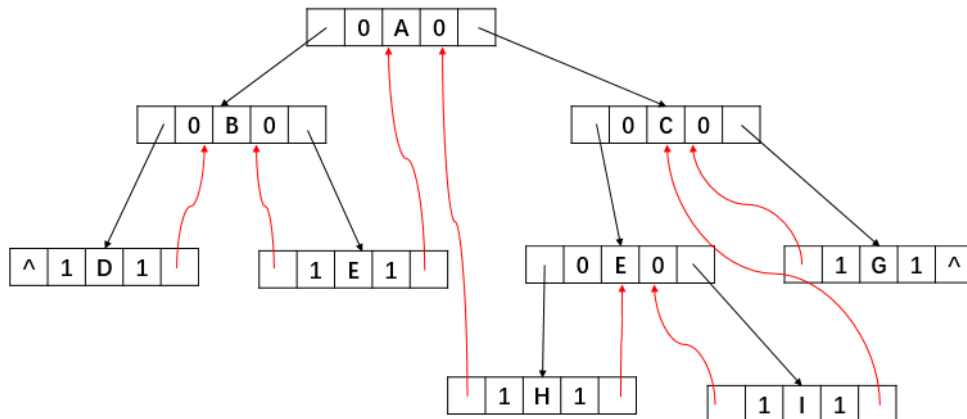


结点存储结构:

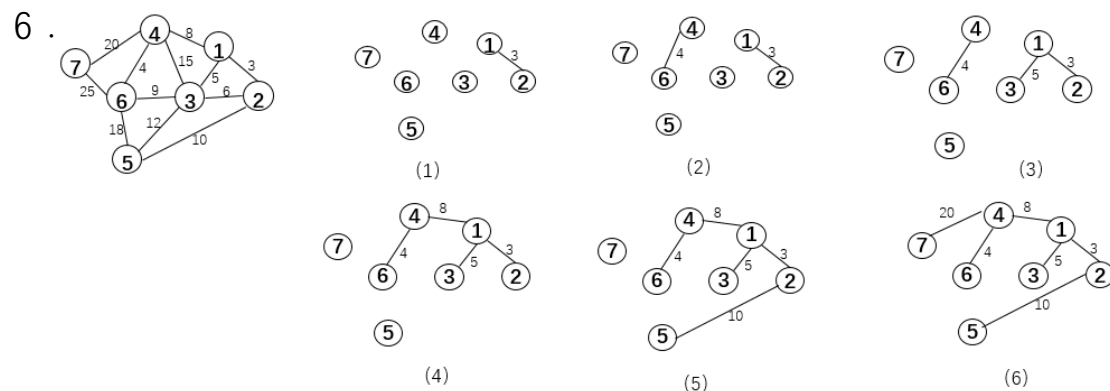
```
typedef struct TBTNode
```

```
{
    char data;
    int ltag, rtag; //左右线索标记, tag=0表示直接指向其孩子结点
    struct TBTNode *lchild;
    struct TBTNode *rchild;
} TBTNode
```

二叉链表:



注: 图中的线索线需用虚线表示



五、算法设计题

```
1. (1) void Insert_x(LinkList &l, int x)
{
    Lnode *p, *s;
    p = l;
    s = (Lnode*)malloc(sizeof(Lnode)); //申请一个新结点s, 使其值域为x
    s->next = NULL;
    s->data = x;
    while(p->next != NULL)
    {
        if(p->next->data == x) //如果找到结点中值大于等于x, 则将s插在该结点之前
        {
            s->next = p->next;
            p->next = s;
        }
        else p = p->next;
    } //while
    p->next = s; //如果链表中所有结点的值都小于x, 则直接将其插在链表最后
}
```

```

(2) Lnode* Find_addr(LinkList L, int key)
{
    Lnode *p;
    p = L->next;
    while(p != NULL)
    {
        if(p->data == key) return;
        else p = p->next;
    }
    return NULL;
}

```

2 . (1)

解一:

```

int n = 0;
void count(BStree *t)
{
    if(t != NULL)
    {
        if(t->lchild == NULL && t->rchild == NULL) ++n;
        count(t->lchild);
        count(t->rchild);
    }
}

```

解二:

```

void count(BStree *t)
{
    int n1, n2;
    if(t == NULL) return 0;
    else if(t->lchild == NULL && t->rchild == NULL) return 1;
    else
    {
        n1 = count(t->lchild);
        n2 = count(t->rchild);
        return n1 + n2;
    }
}

```

(2)

```
int getDepth(BStree *t)
{
    int ld, rd;
    if(t == NULL) return 0; //定义空树的深度为0
    else
    {
        ld = getDepth(t->lchild);
        rd = getDepth(t->rchild);
        return (ld > rd ? ld : rd) + 1;
    }
}
```

3 .

```
void BubbleSort(int A[], int n)
{
    int i, j, flag;
    int temp;
    for(i = n-1; i >= 1; --i)
    {
        flag = 0; //变量flag用来标记本趟排序是否发生了交换
        for(j = 1; j <= i; ++j)
            if(A[j-1] < A[j])
            {
                temp = A[j];
                A[j] = A[j-1];
                A[j-1] = temp;
                flag = 1; //如果发生了交换，则将flag置为1
            }
        if(flag == 0) return; //一趟排序过程中没有发生关键字交换，则证明序列有序，排序结束
    }
}
```